

Metode de sortare — Rezolvări și Teorie

Clasa a IX-a · Profil Matematică-Informatică · Intensiv Informatică

Document destinat profesorului — conține rezolvările complete și note metodologice.

Teorie completă — Metode de sortare

1. Selecția minimului

Idee: împărțim lista în două zone: sortată (stânga) și nesortată (dreapta). La fiecare pas i , găsim minimul din zona nesortată și îl aducem în poziția i prin interschimbare.

Număr de comparații: $n(n-1)/2 \rightarrow O(n^2)$, indiferent de date. **Număr de interschimbări:** exact $n-1$.

Când folosim: când interschimbările sunt costisitoare (ex. fișiere mari pe disc).

2. Metoda bulelor

Idee: comparăm perechi de elemente adiacente și le interschimbăm dacă sunt în ordine greșită. Elementele mari „bulează” spre dreapta.

Optimizare: dacă într-o trecere nu s-a făcut nicio interschimbare, lista e deja sortată $\rightarrow$ oprim.

Cazul favorabil (cu optimizare): $O(n)$. **Cazul nefavorabil:** $O(n^2)$.

3. Sortare cu frecvențe (Counting Sort)

Idee: creăm o listă auxiliară `freq` de dimensiune k . Numărăm aparițiile fiecărei valori, apoi reconstruim lista sortată parcurgând `freq`.

Complexitate: $O(n + k)$ — liniară. Necesită memorie suplimentară $O(k)$.

Restricție: funcționează doar cu valori întregi dintr-un interval cunoscut și relativ mic.

4. Comparație sintetică

Criteriu	Selecție	Bule	Frecvențe
Comparații	$O(n^2)$	$O(n^2)$	—
Interschimbări	$O(n)$	$O(n^2)$	0
Caz favorabil	$O(n^2)$	$O(n)^*$	$O(n+k)$
Memorie extra	$O(1)$	$O(1)$	$O(k)$
Tipuri de date	Orice	Orice	Întregi, interval fix

* Cu optimizarea „oprire dacă fără interschimbări”

5. Greșeli frecvente ale elevilor

- Selecție:** interschimbarea greșită — atribuire directă în loc de swap cu variabilă temporară.
- Bule:** bucla interioară nu scade la fiecare trecere, deci compară și elemente deja sortate.
- Frecvențe:** indexare greșită în lista de frecvențe (confuzie între valoare și indice).
- Toate:** iterarea `range(n)` vs `range(n-1)` — off-by-one errors.

SIMPLU Exercițiile 1-4 · 5 p / exercițiu

EXERCIȚIUL 1 (5P)

Urmărire selecție minimului pe [5, 2, 8, 1, 9, 3] — completare tabel.

REZOLVARE — TABEL COMPLET

Pas	lst[0]	lst[1]	lst[2]	lst[3]	lst[4]	lst[5]	Min la poz.
Start	5	2	8	1	9	3	—
$i=0$	1	2	8	5	9	3	3
$i=1$	1	2	8	5	9	3	1 (deja ok)
$i=2$	1	2	3	5	9	8	5
$i=3$	1	2	3	5	9	8	3 (deja ok)
$i=4$	1	2	3	5	8	9	5
Final	1	2	3	5	8	9	—

EXERCIȚIUL 2 (5P)

O trecere completă metoda bulelor pe [4, 7, 2, 9, 1].

REZOLVARE — TABEL COMPLET

Comparare	lst[0]	lst[1]	lst[2]	lst[3]	lst[4]	Interschimbare?
Start	4	7	2	9	1	—
lst[0]↔lst[1]	4	7	2	9	1	Nu (4 < 7)
lst[1]↔lst[2]	4	2	7	9	1	Da (7 > 2)
lst[2]↔lst[3]	4	2	7	9	1	Nu (7 < 9)
lst[3]↔lst[4]	4	2	7	1	9	Da (9 > 1)

După o trecere: [4, 2, 7, 1, 9]. Elementul **9** (cel mai mare) ajunge la locul lui. La fiecare trecere, maximul din subșirul nesortat ajunge la capătul drept.

EXERCIȚIUL 3 (5P)

Lista de frecvențe pentru [7, 9, 8, 7, 10, 9, 7, 8, 6, 9].

REZOLVARE

Valoare	1	2	3	4	5	6	7	8	9	10
Frecvență	0	0	0	0	0	1	3	2	3	1

LISTA SORTATĂ RECONSTRUITĂ

[6, 7, 7, 7, 8, 8, 9, 9, 9, 10]

Reconstrucție: parcurgem valorile 1..10; pentru fiecare valoare v cu frecvență f , adăugăm f apariții ale lui v în lista rezultat. Valorile cu frecvență 0 nu contribuie.

EXERCIȚIUL 4 (5P)

Asociere afirmații cu metode: S = Selecție, B = Bule, F = Frecvențe.

REZOLVARE

- a) F** — Frecvențele necesită lista auxiliară de dimensiunea intervalului [val_min, val_max].
- b) B** — La metoda bulelor, la fiecare trecere, elementul maxim „urcă” prin interschimbări spre capătul drept.
- c) S** — Selecția minimului face exact $n-1$ interschimbări (câte una per pas), indiferent de ordinea inițială.
- d) F** — Sortarea cu frecvențe este $O(n+k)$; pentru note întregi 1-10, $k=10$ fix $\rightarrow O(n)$.
- e) B** — Optimizarea metodei bulelor: dacă o trecere completă nu produce nicio interschimbare, algoritmul se oprește.

Discuție: De ce selecția face exact $n-1$ interschimbări? La fiecare pas i plasăm definitiv un element $\rightarrow n-1$ pași, câte o interschimbare fiecare.

MEDIU

Exercițiile 5-8 · 10 p / exercițiu

EXERCIȚIUL 5 (10P)

Implementare `sortare_selectie(lst)` cu afișare după fiecare interschimbare. Testare cu `[64, 25, 12, 22, 11]`.

REZOLVARE

```
def sortare_selectie(lst):
    n = len(lst)
    for i in range(n - 1):
        # ultimul element se sortează automat
        poz_min = i
        for j in range(i + 1, n):
            # căutăm min în zona nesortată
            if lst[j] < lst[poz_min]:
                poz_min = j
        if poz_min != i:
            # interschimbăm doar dacă e necesar
            lst[i], lst[poz_min] = lst[poz_min], lst[i]
            print(f"Pas {i+1}: {lst}")

numere = [64, 25, 12, 22, 11]
print(f"Inițial: {numere}")
sortare_selectie(numere)
print(f"Sortat: {numere}")
```

OUTPUT

Inițial: [64, 25, 12, 22, 11] Pas 1: [11, 25, 12, 22, 64] Pas 2: [11, 12, 25, 22, 64] Pas 3: [11, 12, 22, 25, 64] Sortat: [11, 12, 22, 25, 64]

Detalii cheie: bucla exterioară merge până la `n-1`, bucla interioară pornește de la `i+1`. Interschimbarea simultană `a, b = b, a` în Python nu necesită variabilă temporară.

Pas 4 nu apare deoarece minimul din `[25, 64]` este 25, care e deja la locul lui (`poz_min == i`).

EXERCIȚIUL 6 (10P)

Implementare `sortare_bule(lst)` cu optimizare de oprire. Testare cu `[5, 1, 4, 2, 8]`.

REZOLVARE

```
def sortare_bule(lst):
    n = len(lst)
    for i in range(n - 1):
        interschimb = False
        # flag pentru optimizare
        for j in range(n - 1 - i):
            # ultimele i elem. sunt deja sortate
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                interschimb = True
        print(f"Trecerea {i+1}: {lst}")
    if not interschimb:
        # nicio schimbare → gata
        print("Lista deja sortată — oprire anticipată.")
        break

numere = [5, 1, 4, 2, 8]
print(f"Inițial: {numere}")
sortare_bule(numere)
print(f"Sortat: {numere}")
```

OUTPUT

Inițial: [5, 1, 4, 2, 8] Trecerea 1: [1, 4, 2, 5, 8] Trecerea 2: [1, 2, 4, 5, 8] Trecerea 3: [1, 2, 4, 5, 8] Lista deja sortată — oprire anticipată. Sortat: [1, 2, 4, 5, 8]

Optimizare cheie: `n - 1 - i` în bucla interioară — după trecerea *i*, ultimele *i* elemente sunt deja la locul lor și nu mai trebuie comparate.

EXERCIȚIUL 7 (10P)

Implementare `sortare_frecvente(lst, val_min, val_max)` . Testare cu note 1-10.

REZOLVARE

```
def sortare_frecvente(lst, val_min, val_max):
    # Pasul 1: inițializăm lista de frecvențe cu zerouri
    k = val_max - val_min + 1
    freq = [0] * k

    # Pasul 2: numărăm aparițiile fiecărei valori
    for val in lst:
        freq[val - val_min] += 1    # translatăm valoarea în indice

    # Pasul 3: reconstruim lista sortată
    rezultat = []
    for i in range(k):
        rezultat.extend([i + val_min] * freq[i])

    return rezultat

note = [7, 9, 8, 7, 10, 9, 7, 8, 6, 9]
print(f"Note inițiale: {note}")
sortate = sortare_frecvente(note, 1, 10)
print(f"Note sortate: {sortate}")
```

OUTPUT

Note inițiale: [7, 9, 8, 7, 10, 9, 7, 8, 6, 9] Note sortate: [6, 7, 7, 7, 8, 8, 9, 9, 9, 10]

EXERCIȚIUL 8 (10P)

Identifică cele trei greșeli din codul de selecție dat.

```
def sortare_selectie(lst):
    n = len(lst)
    for i in range(n):
        poz_min = i
        for j in range(i, n):
            if lst[j] < lst[poz_min]:
                poz_min = j
        lst[i] = lst[poz_min]
        lst[poz_min] = lst[i]
```

REZOLVARE — CELE TREI GREȘELI

Greșeala 1 — bucla exterioară: `range(n)` în loc de `range(n-1)`

La ultimul pas ($i=n-1$), bucla interioară rulează pe un singur element → inefficient. Corect: `range(n-1)` .

Greșeala 2 — bucla interioară: `range(i, n)` în loc de `range(i+1, n)`

Pornim de la i → comparăm elementul cu el însuși. Ineficient conceptual. Corect: `range(i+1, n)` .

Greșeala 3 — interschimbare greșită (FATALĂ):

`lst[i] = lst[poz_min]` suprascrie valoarea lui `lst[i]` înainte de a o salva. `lst[poz_min] = lst[i]` copiază acum valoarea deja suprascrisă → valoarea originală se pierde.

```
# Cod corectat:
def sortare_selectie(lst):
    n = len(lst)
    for i in range(n - 1):          # Fix 1: n-1
        poz_min = i
        for j in range(i + 1, n):   # Fix 2: i+1
            if lst[j] < lst[poz_min]:
                poz_min = j
        lst[i], lst[poz_min] = lst[poz_min], lst[i] # Fix 3: swap corect
```

EXERCIȚIUL 9 (15P)

Sortare descrescătoare scoruri (selecția maximului) + podium. Testare cu [1200, 850, 2300, 400, 1750, 980, 3100, 620] .

REZOLVARE

```
def sortare_selectie_desc(lst):
    n = len(lst)
    for i in range(n - 1):
        poz_max = i          # căutăm maximul, nu minimul
        for j in range(i + 1, n):
            if lst[j] > lst[poz_max]:    # ">" în loc de "<"
                poz_max = j
        lst[i], lst[poz_max] = lst[poz_max], lst[i]

scoruri = [1200, 850, 2300, 400, 1750, 980, 3100, 620]
sortare_selectie_desc(scoruri)

print("PODIUM")
medalii = ["Aur", "Argint", "Bronz"]
for i in range(3):
    print(f"{medalii[i]}: {scoruri[i]} puncte")
print(f"\nToate scorurile ordonate: {scoruri}")
```

OUTPUT

```
PODIUM Aur: 3100 puncte Argint: 2300 puncte Bronz: 1750 puncte Toate scorurile ordonate: [3100, 2300, 1750, 1200, 980, 850, 620, 400]
```

Singura modificare față de selecția minimului: comparăm cu `>` și căutăm `poz_max`. La fiecare pas plasăm definitiv cel mai mare element rămas.

EXERCIȚIUL 10 (15P)

Triaj medical — sortare descrescătoare cu frecvențe (gravitate 1-5). Testare cu [3, 1, 5, 2, 1, 4, 2, 3, 1, 5, 2] .

REZOLVARE

```
def triaj_medical(pacienti, val_min, val_max):
    k = val_max - val_min + 1
    freq = [0] * k

    for gravitate in pacienti:
        freq[gravitate - val_min] += 1

    # Reconstruim crescător, apoi inversăm (1 = cel mai urgent)
    ordine = []
    for v in range(val_min, val_max + 1):
        ordine.extend([v] * freq[v - val_min])
    ordine.reverse()
    return ordine, freq

pacienti = [3, 1, 5, 2, 1, 4, 2, 3, 1, 5, 2]
ordine, freq = triaj_medical(pacienti, 1, 5)

print("Ordine consultare (1=cel mai urgent):")
for nr, gravitate in enumerate(ordine, 1):
    print(f" Pacientul {nr}: gravitate {gravitate}")
print(f"\nDistribuție: {freq} (gravități 1-5)")
```

OUTPUT

```
Ordine consultare (1=cel mai urgent): Pacientul 1: gravitate 1 Pacientul 2: gravitate 1 Pacientul 3: gravitate 1 Pacientul 4: gravitate 2 Pacientul 5: gravitate 2 Pacientul 6: gravitate 2 Pacientul 7: gravitate 3 Pacientul 8: gravitate 3 Pacientul 9: gravitate 4 Pacientul 10: gravitate 5 Pacientul 11: gravitate 5 Distribuție: [3, 3, 2, 1, 2] (gravități 1-5)
```

EXERCITIUL 11 (15P)

Comparare bule vs. selecție — numărare exactă comparații și interschimbări pe [9, 7, 5, 3, 1, 2, 4, 6, 8].

REZOLVARE

```
def bule_cu_contor(lst):
    lst = lst[:] # copie — nu modificăm originalul
    n = len(lst)
    comparari = interschimbări = 0
    for i in range(n - 1):
        interschimbat = False
        for j in range(n - 1 - i):
            comparari += 1
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                interschimbări += 1
                interschimbat = True
        if not interschimbat:
            break
    return comparari, interschimbări

def selectie_cu_contor(lst):
    lst = lst[:]
    n = len(lst)
    comparari = interschimbări = 0
    for i in range(n - 1):
        poz_min = i
        for j in range(i + 1, n):
            comparari += 1
            if lst[j] < lst[poz_min]:
                poz_min = j
        if poz_min != i:
            lst[i], lst[poz_min] = lst[poz_min], lst[i]
            interschimbări += 1
    return comparari, interschimbări

date = [9, 7, 5, 3, 1, 2, 4, 6, 8]
cb, ib = bule_cu_contor(date)
cs, is_ = selectie_cu_contor(date)
print(f"Bule:      {cb} comparații, {ib} interschimbări")
print(f"Selecție: {cs} comparații, {is_} interschimbări")
```

OUTPUT

Bule: 36 comparații, 20 interschimbări Selecție: 36 comparații, 7 interschimbări

Interpretare: Ambele fac același număr de comparații ($36 = n(n-1)/2$ pentru $n=9$). Diferența majoră: selecția face doar **7 interschimbări** față de **20** la bule. Concluzie: selecția este mai eficientă când interschimbările sunt costisitoare; bule poate fi mai rapidă pe liste aproape sortate datorită optimizării de oprire anticipată.

EXERCITIUL 12 (15P)

Clasament concurs matematică cu perechi (nume, punctaj), ex-aequo, sortare descrescătoare.

REZOLVARE — CU JUSTIFICAREA ALEGERII METODEI

Alegerea metodei: Punctajele sunt întregi în intervalul 0-100 → sortare cu frecvențe, $O(n+100) \approx O(n)$. Dar trebuie să păstrăm asocierea nume-punctaj → folosim **selecția maximului adaptată** pentru a sorta lista de perechi descrescător după punctaj.

```
def sortare_clasament(concurenti):
    # concurenti = lista de tuple (nume, punctaj)
    n = len(concurenti)
    for i in range(n - 1):
        poz_max = i
        for j in range(i + 1, n):
            if concurenti[j][1] > concurenti[poz_max][1]:
                poz_max = j
        concurenti[i], concurenti[poz_max] = concurenti[poz_max], concurenti[i]

def afiseaza_clasament(concurenti):
    print(f"{'Loc':<5} {'Nume':<12} {'Punctaj'}")
    print("-" * 28)
    loc = 1
    for i, (nume, punctaj) in enumerate(concurenti):
        if i > 0 and punctaj < concurenti[i - 1][1]:
            loc = i + 1
        print(f"{loc:<5} {nume:<12} {punctaj}")

participanti = [
    ("Ana", 87), ("Bogdan", 92), ("Carla", 87),
    ("Dan", 75), ("Elena", 92)
]
sortare_clasament(participanti)
afiseaza_clasament(participanti)
```

OUTPUT

```
Loc Nume Punctaj ----- 1 Bogdan 92 1 Elena 92 3 Ana 87 3 Carla 87 5 Dan 75
```